

E-STORE

Application e-commerce full-stack

Module	Full-Stack
Encadrant	Pr. Omar Zahour
Etablissement	FSBM - Universite Hassan II
Annee	2025-2026
Auteurs	Akram Belmoussa & Nouhaila Ben Soumane

Plan de la presentation

1. Contexte et objectifs du projet
2. Architecture (3 couches x 5 domaines)
3. Stack technique
4. Backend Spring Boot - exemples de code
5. Securite (Spring Security + JWT)
6. MongoDB pour les avis (persistance hybride)
7. Frontend Angular 17
8. Demonstration en direct
9. Tests, DevOps et difficulte Docker
10. Conclusion et perspectives

1. Contexte et objectifs

Mini-projet pedagogique e-commerce

Application complete : inscription, catalogue, panier, commandes, avis

Objectif 1 : maitriser une architecture full-stack moderne

Objectif 2 : illustrer la persistance hybride SQL + NoSQL

Objectif 3 : appliquer les bonnes pratiques (DTOs, transactions, tests)

Objectif 4 : securiser via JWT + Spring Security 6

Objectif 5 : produire une UI moderne avec Angular 17 standalone

2. Architecture en 3 couches x 5 domaines

3 couches techniques

PRESENTATION (Angular)	Composants standalone, services HTTP, formulaires
LOGIQUE METIER (Spring Boot)	Controllers REST, Services @Transactional, DTOs
ACCES AUX DONNEES	Spring Data JPA + Spring Data MongoDB

6 sous-packages domaine

customer	User, Profile, JWT, Auth
catalog	Categorie, Produit, recherche
inventory	Stock par produit
shopping	Cart, CartItem
billing	Order, OrderItem (transactionnel)
review	Avis (MongoDB)

3. Stack technique

Couche	Technologies
Frontend	Angular 17 standalone, Bootstrap 5, RxJS, TypeScript 5
Backend	Spring Boot 3.3, Spring Security 6, JWT (jjwt 0.12)
ORM / Donnees	Spring Data JPA + Hibernate, Spring Data MongoDB
Bases	MySQL 8 (prod) / H2 (dev) + MongoDB 7
Build	Maven 3.9, npm, Angular CLI
Tests	JUnit 5 + Mockito (9 tests, BUILD SUCCESS)
DevOps	Docker Compose : MySQL + Mongo + phpMyAdmin + mongo-express

4. Backend - exemple : checkout transactionnel

OrderService.checkout() - atomicite ACID

```
@Transactional
public OrderDto checkout() {
    Cart cart = cartService.getOrCreateCart(user);
    if (cart.getItems().isEmpty())
        throw new BusinessException("Votre panier est vide");

    // 1) Verifier le stock pour TOUS les items
    for (CartItem ci : cart.getItems())
        inventoryService.checkAvailability(ci.getProduct().getId(), ci.getQuantity());

    // 2) Creer la commande + decremener stock
    Order order = Order.builder().user(user).status(PENDING).build();
    for (CartItem ci : cart.getItems()) {
        order.getItems().add(OrderItem.builder()...build());
        inventoryService.decrement(ci.getProduct().getId(), ci.getQuantity());
    }
    order.setStatus(CONFIRMED);
    Order saved = orderRepository.save(order);

    // 3) Vider le panier
    cartService.clearCart(cart);
    return OrderDto.from(saved);
}
```

5. Securite - JWT + Spring Security 6

1	Client envoie email + password	<code>POST /api/auth/login</code>
2	AuthService valide via BCrypt	<code>passwordEncoder.matches(...)</code>
3	JwtService genere token signe HS256	<code>Jwts.builder().signWith(key)</code>
4	Client stocke le token	<code>localStorage 'estore.token'</code>
5	Toute requete : Bearer <token>	<code>AuthInterceptor</code>
6	JwtFilter valide a chaque appel	<code>JwtAuthenticationFilter</code>

- **BCrypt** pour les mots de passe (cout 10).
- **JWT HS256**, expiration 24h, claims : uid, role, name.
- **Endpoints publics** : `/api/auth/**`, `GET /products /categories /reviews`.
- **@PreAuthorize("hasRole('ADMIN')")** sur les operations d'administration.

6. MongoDB pour les avis (persistance hybride)

Pourquoi un second SGBD ?

Avis = donnees semi-structurees, volumineuses, en lecture frequente

Pas de relations strictes - schema flexible

Spring Data MongoDB : meme abstraction que JPA

Repository : `findByProductIdOrderByCreatedAtDesc(productId)`

Document type :

{ productId, userId, authorName, rating(1-5), comment, createdAt }

Comportement defensif : si Mongo absent, l'app continue (warning)

7. Frontend Angular 17

Composants standalone + signals

Standalone components (pas de NgModule) - moderne et concis

Lazy loading par route : bundle initial 13 ko + features a la demande

Signals pour l'etat reactif : `cart.itemCount()` dans le header

AuthService : BehaviorSubject + localStorage persistance

AuthInterceptor : ajoute Bearer a chaque requete

ErrorInterceptor : gestion centralisee 401/4xx/0 (toast)

AuthGuard sur /cart, /orders, /profile

Bootstrap 5 + thematique cohesive, responsive

8. Demonstration en direct

Scenario de demonstration en 8 etapes

1. Catalogue : 12 produits, recherche, filtre par categorie
2. Fiche produit : prix, stock, avis
3. Tentative panier sans connexion -> redirection /login
4. Connexion user@estore.ma / User@123
5. Ajout de 2 produits au panier (badge passe a 2)
6. Panier : modification quantite, total recalcule
7. Validation commande -> /orders, commande #1 confirmee
8. Depot d'un avis 5 etoiles sur un produit

9. Tests, DevOps et la difficulté Docker

Tests unitaires

- ProductServiceTest (4 tests)
- CartServiceTest (2 tests)
- OrderServiceTest (3 tests)
- TOTAL : 9 tests, 0 failure
- Mockito @MockitoSettings(LENIENT)

Difficulté Docker rencontrée

- Erreur : 'C:\\ProgramData\\DockerDesktop must be owned
- by an elevated account'
- Cause : dossier résiduel d'une installation précédente
- Solution 1 : rmdir /s /q en admin + réinstallation
- Solution 2 : profil dev (H2 in-memory) - pas d'installation
- => Architecture pensée pour 2 modes dès le départ

10. Conclusion et perspectives

Bilan

- [+] Architecture 3 couches x 5 domaines respectee
- [+] Securite robuste (JWT + BCrypt + @PreAuthorize)
- [+] Persistance hybride SQL + NoSQL operationnelle
- [+] 9 tests unitaires verts, build reproductible

Perspectives :

- Paiement en ligne (Stripe / CMI)
- Notifications (emails de confirmation, SMS)
- Recommandations produits (ML)
- Deploiement cloud (Heroku, AWS, Azure)

Merci pour votre attention - Questions ?